

517编程新年挑战赛

-----祝大家新年快乐! -----

A 517的宝贵经验：一段（条件判断，数据类型）

B 破译信息：二段（数位分解）

C 买年货：三段（数组循环应用综合）

D 世界杯：三段+（暴力模拟）

E 陆游器：四段（递归搜索）

F 程序天梯赛：四段（二分）

G 时速派送：五段（区间DP）

H 大富翁 五段：（最短路）

I 放羊 六-七段：（树上贪心）

J 线段值 八段：（线段树）

A-517的宝贵经验

注意审题，输入的是实数。因此需要用double定义，然后根据题目要求判断即可。

```
double a,b,c;
cin>>a>>b>>c;
if(a+b==c) cout<<"YES";
else if(a+c==b) cout<<"YES";
else if(c+b==a) cout<<"YES";
else if(a*b==c) cout<<"YES";
else if(b*c==a) cout<<"YES";
else if(a*c==b) cout<<"YES";
else cout<<"NO";
```

B-破译信息

直接将输入的 n 进行数位分解，并记录下 n 有多少位，按照题目要求计算即可

```

void solve(){
    cin>>n;
    int cnt=0,ans=0;
    while(n){
        cnt++;
        ans-=n%10;
        n/=10;
    }
    ans+=cnt/2*cnt;
    cout<<ans;
}

```

C-买年货

按照题目要求，求出每个收银台的排队时间，取最小值输出即可

```

int n,m;
int g[100010];
int main(){
    cin>>n;
    int ans=1e9;
    for(int i=0;i<n;i++){
        cin>>g[i];
    }
    for(int i=0;i<n;i++){
        int res=0;
        for(int j=0;j<g[i];j++){
            int x; cin>>x;
            res+=x*5+15;
        }
        ans=min(ans,res);
    }
    cout<<ans;
    return 0;
}

```

D-世界杯

思路一（模拟-三段）：

对每个队员进行分类讨论：

1. 只能担任一种位置。就让他干这个位置
2. 可以担任两种位置。就暴力枚举哪种情况更优。
3. 三种位置都可以担任。哪里少人就补哪

```
#include <bits/stdc++.h>
```

```

using namespace std;
typedef long long ll;
int o[15], v[15], e[15], c[10];
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie (0);
    cout.tie (0);
    int n;
    cin >> n;
    for (int i = 1; i <= n; i ++ )
    {
        char c;
        cin >> o[i] >> c >> v[i] >> c >> e[i];
    }
    int m;
    cin >> m;
    for (int i = 1; i <= m; i ++ )
    {
        string s;
        cin >> s;
        if(s == "O")
        {
            c[1]++;
        }
        else if(s == "V")
        {
            c[2]++;
        }
        else if(s == "N")
        {
            c[3]++;
        }
        else if(s=="NO" || s=="ON")
        {
            c[4]++;
        }
        else if(s == "NV" || s == "VN")
        {
            c[5]++;
        }
        else if(s == "OV" || s == "VO")
        {
            c[6]++;
        }
        else
        {
            c[7]++;
        }
    }
}

```

```

for (int i = 1; i <= n; i ++)
{
    bool f = 0;
    for(int j = 0; j <= c[4]; j ++)
    {
        for(int k = 0; k <= c[5]; k ++)
        {
            for(int l = 0; l <= c[6]; l ++)
            {
                int sum = 0;
                int s1 = c[4] - j, s2 = c[5] - k, s3 = c[6] - l;
                if(o[i] > c[1] + s1 + l)
                {
                    sum += o[i] - (c[1] + l + s1);
                }
                if(v[i] > c[2] + s2 + s3)
                {
                    sum += v[i] - (c[2] + s2 + s3);
                }
                if(e[i] > c[3] + j + k)
                {
                    sum += e[i] - (c[3] + j + k);
                }
                if(sum <= c[7])
                {
                    f=1;
                    break;
                }
            }
        }
        if(f == 1)
        {
            break;
        }
    }
    if(f == 1)
    {
        break;
    }
}
if(f == 1)
{
    cout << "YES\n";
}
else
{
    cout << "NO\n";
}
}
return 0;

```

```
}
```

思路二(搜索思想-四段):

将每个队友可以选择的位置记录下来, 直接 *DFS* 即可, 为了保证不重, 我们可以强制让每次选择的队员编号比之前选择的大, 之后暴力搜索有哪些可以组成的方案并打上标记, 之后 $O(1)$ 回答即可。

```
include<bits/stdc++.h>
using namespace std;
int n,m,t[4][11];
bool v[23][4],flag,ans[11][11][11];
string s;
void dfs(int k,int g,int t1,int t2,int t3){
    if(k==11){
        ans[t1][t2][t3]=1;
        return;
    }
    for(int i=g;i<=m;i++){
        for(int j=1;j<=3;j++){
            if(!v[i][j]) continue;
            if(j==1) dfs(k+1,i+1,t1+1,t2,t3);
            if(j==2) dfs(k+1,i+1,t1,t2+1,t3);
            if(j==3) dfs(k+1,i+1,t1,t2,t3+1);
        }
    }
}
int main(){
    scanf("%d",&n);
    for(int i=1;i<=n;i++) scanf("%d-%d-%d",&t[1][i],&t[2][i],&t[3][i]);
    scanf("%d",&m);
    for(int i=1;i<=m;i++){
        cin>>s;
        for(int j=0;j<s.size();j++){
            if(s[j]=='O') v[i][1]=1;
            if(s[j]=='N') v[i][3]=1;
            if(s[j]=='V') v[i][2]=1;
        }
    }
    dfs(1,1,0,0,0);
    for(int i=1;i<=n;i++){
        if(ans[t[1][i]][t[2][i]][t[3][i]]) printf("YES\n");
        else printf("NO\n");
    }
    return 0;
}
```

E-陆游器

方法一：

如果 $(x_1, y_1), (x_2, y_2)$ 在一个连通块内，答案为0

否则就要利用陆游器：

使用**bfs**或**dfs**找出 (x_1, y_1) 和 (x_2, y_2) 所在的联通块 A, B ，并记录下 A, B 中的所有点

因为要最小化答案，数据范围很小，可以双重循环处理。

使用双重循环枚举 A, B 中的所有点 A_i, B_j ，每次计算从 A_i 到 B_j 的花费，取最小值输出即可

```
#include<bits/stdc++.h>
#define int long long
using namespace std;
const int N=51;
int n;
int sx,sy,ex,ey;
char s[N][N];
vector<pair<int,int>>ve1,ve2;
bool vis[N][N];
int dx[]={0,0,1,-1};
int dy[]={1,-1,0,0};
void bfs(int x,int y,int f){
    memset(vis,0,sizeof vis);
    queue<pair<int,int>>q;
    q.push({x,y}); vis[x][y]=1;
    while(q.size()){
        int xx=q.front().first;
        int yy=q.front().second;
        q.pop();
        if(f==1) ve1.push_back({xx,yy});
        else ve2.push_back({xx,yy});
        for(int i=0;i<4;i++){
            int a=xx+dx[i],b=yy+dy[i];
            if(a<1||b<1||a>n||b>n) continue;
            if(vis[a][b]||s[a][b]=='1') continue;
            vis[a][b]=1;
            q.push({a,b});
        }
    }
}
int get(int a,int b){
    return (a-b)*(a-b);
}
signed main(){
    cin>>n>>sx>>sy>>ex>>ey;
    for(int i=1;i<=n;i++) cin>>s[i]+1;
    bfs(sx,sy,1);
```

```

    bfs(ex,ey,2);
    int ans=1e9;
    for(int i=0;i<ve1.size();i++){
        for(int j=0;j<ve2.size();j++){
            int num=get(ve1[i].first,ve2[j].first)+get(ve1[i].second,ve2[j].second);
            ans=min(ans,num);
        }
    }
    cout<<ans;
    return 0;
}

```

方法二:

我们考虑两种获得最小花费的做法:

1. 直接从陆地上走过去
2. 到达某个点，然后使用陆游器传送到另一个点，最后到达终点

第一种方法很简单，是“染色问题”的模板。

然而第二种就要动脑筋了。

考虑到数据范围不大，所以我们这样做：

- 从起点开始深搜，如果搜到终点直接走人。同时记录到达的点。
- 从终点开始深搜，并且搜到一个点就和前面的点求 *dis*，并求最小。

总复杂度最坏也才 $O(N^2)$ ，绰绰有余。

```

#include <bits/stdc++.h>
using namespace std;
const int dx[]={1,0,-1,0},dy[]={0,1,0,-1}; //方向增量数组
bool v[101][101],v2[101][101]; //访问标记数组
int l[10005],r[10005]; //记录点
int cnt,n,fx,fy,sx,sy,ans(1<<30); //最大值
char ch;
void dfs(int x,int y){
    v[x][y]=true; //标记一下
    l[++cnt]=x;r[cnt]=y; //记录这个点
    if(x==fx&&y==fy){ //到终点直接走人
        puts("0");
        exit(0);
    }
    for(int i=0;i<4;++i){
        x+=dx[i];y+=dy[i]; //方向增量,
        if(1<=x&&x<=n&&1<=y&&y<=n&&!v[x][y])
            dfs(x,y);
        x-=dx[i];y-=dy[i]; //变回来
    }
}

```

```

    }
    return;
}

void dfs2(int x,int y){
    v2[x][y]=true;
    int dis;
    for(int i=1;i<=cnt;++i){
        dis=(x-l[i])*(x-l[i])+(y-r[i])*(y-r[i]); //算和之前cnt个搜到的点的最小距离
        if(dis<ans)
            ans=dis; //并更新
    }
    for(int i=0;i<4;++i){
        x+=dx[i];y+=dy[i];
        if(1<=x&& x<=n&& 1<=y&& y<=n&& !v2[x][y])
            dfs2(x,y);
        x-=dx[i];y-=dy[i];
    }
    return;
}

int main(){
    int i,j;
    scanf("%d %d %d %d %d",&n,&sx,&sy,&fx,&fy);
    for(i=1;i<=n;++i)
        for(j=1;j<=n;++j){
            cin>>ch; //不用cin要scanf("%s"), 数据不大就偷个懒
            v2[i][j]=v[i][j]=ch=='1'; //不能走就标记走过了
        }
    dfs(sx,sy); //从起点搜
    dfs2(fx,fy); //从终点搜
    printf("%d",ans);
    return 0;
}

```

F-程序天梯赛

为了让时间尽可能小，所以应该让每名成员尽可能做自己擅长的题目

假设最短时间为 ans ，则对于时间 t

- $ans \leq t$ ，一定可以让成员完成所有题目
- $ans > t$ ，一定不能完成全部题目

我们先处理出每个成员擅长的题目数量，即用桶 $cnt[i]$ 记录第 i 名成员擅长的题目数量。

考虑二分答案，假设当前枚举到的时间为 mid

枚举每位成员 i

- 如果 $cnt[i] > mid$ ，说明第 i 名成员应该将剩下的 $cnt[i] - mid$ 道题目交给其他成员完成

- 如果 $cnt[i] < mid$, 说明第 i 名成员可以多做 $(mid - cnt[i]) / 2$ 道自己不擅长的题目
- 最后判断是否有余下的题目即可

```
#include<bits/stdc++.h>
#define int long long
using namespace std;
const int N=2e5+10;
int n,m;
int t[N],now[N];
bool check(int mid){
    for(int i=1;i<=n;i++) now[i]=t[i];
    int res=0;
    for(int i=1;i<=n;i++){
        if(now[i]>mid) res+=now[i]-mid;
    }
    for(int i=1;i<=n;i++){
        if(now[i]<mid) res-=(mid-now[i])/2;
    }
    return res<=0;
}
signed main(){
    cin>>n>>m;
    for(int i=0;i<m;i++){
        int x; cin>>x;
        t[x]++;
    }
    int l=1,r=2*m,ans=r;
    while(l<=r){
        int mid=l+r>>1;
        if(check(mid)) ans=mid,r=mid-1;
        else l=mid+1;
    }
    cout<<ans;
    return 0;
}
```

G—时速派送

因为每次取的都是极差，考虑当选 x 时对 d 的影响

- 如果 x 为最大值，则 $d = x - min$
- 如果 x 为最小值，则 $d = max - x$
- 其他情况， $d = max - min$

贪心的考虑，每次我们应该选择更新最大最小值，即选择的区间是连续的，因为如果不选择更新最大或最小值，此时 d 会选择一个相对大的值

先对 a_i 进行排序，进行区间 dp

$f[l][r]$ 表示选择区间 $[l,r]$ 的最小值

状态转移

$f[l][r]=a[r]-a[l]+\min(f[l+1][r],f[l][r-1]);$

```
#include<bits/stdc++.h>
#define int long long
using namespace std;
const int N=2010;
int n,m;
int g[N];
int f[N][N],s[N][N];
signed main(){
    cin>>n;
    for(int i=1;i<=n;i++){
        cin>>g[i];
        assert(g[i]>=1&&g[i]<=(int)1e9);
    }
    sort(g+1,g+n+1);

    for(int len=2;len<=n;len++){
        for(int l=1;l<=n-len+1;l++){
            int r=l+len-1;
            f[l][r]=g[r]-g[l]+min(f[l][r-1],f[l+1][r]);
        }
    }
    cout<<f[1][n];
    return 0;
}
```

H-大富翁

最短路问题

因为题目要求一次移动三步的最小移动次数，所以可以定义

$dist[i][0,1,2]=d$

走到 i 点恰好移动了 $d\%3=0,1,2$ 次的最短距离

再利用堆优化迪杰斯特拉求最短路即可

```
while(q.size()){
    int d=q.top().first,u=q.top().second;
    q.pop();

    if(vis[u][d%3]) continue;
    vis[u][d%3]=1;
```

```

        for(int i=0;i<ve[u].size();i++){
            int j=ve[u][i];
            if(dist[j][(d+1)%3]>dist[u][d%3]+1){//d=dist[u][d%3]
                dist[j][(d+1)%3]=dist[u][d%3]+1;
                q.push({dist[j][(d+1)%3],j});
            }
        }
    }
}

if(dist[t][0]==0x3f3f3f3f) cout<<-1;//当走到t且恰好走了%3=0次时未被更新
else cout<<dist[t][0]/3;

```

```

#include<bits/stdc++.h>
using namespace std;
typedef pair<int,int>PII;
const int N=1e5+10;

vector<int>ve[N];
int dist[N][3];
bool vis[N][3];
int n,m,s,t;

void djs(){
    memset(dist,0x3f,sizeof dist);
    dist[s][0]=0;
    priority_queue<PII,vector<PII>,greater<PII>>q;
    q.push({0,s});
    while(q.size()){
        int d=q.top().first,u=q.top().second;
        q.pop();

        if(vis[u][d%3]) continue;
        vis[u][d%3]=1;

        for(int i=0;i<ve[u].size();i++){
            int j=ve[u][i];
            if(dist[j][(d+1)%3]>dist[u][d%3]+1){
                dist[j][(d+1)%3]=dist[u][d%3]+1;
                q.push({dist[j][(d+1)%3],j});
            }
        }
    }

    if(dist[t][0]==0x3f3f3f3f) cout<<-1;
    else cout<<dist[t][0]/3;
}

int main(){

    cin>>n>>m;

```

```

while(m--){
    int a,b; cin>>a>>b;
    ve[a].push_back(b);
}
cin>>s>>t;

djs();
return 0;
}

```

I-放羊

考虑贪心：

对于深度最深的一个羊点，选择它能选择的的最高的祖宗一定比选择它的某个兄弟结点更优，因为这样才能看守最多的羊。

根据上述贪心策略，可以先预处理出每个点距离羊点的最短距离，再对羊点按照深度从小到大排序，从每个羊点暴力向上搜到能选择的深度最低的祖宗结点，并判断标记这个祖宗的子树中是否有其他羊点被这个祖宗结点看守

因为每条路径最多走一次，所以均摊复杂度为 $O(n)$ ，排序复杂度 $O(n\log n)$ ，所以总复杂度 $O(n\log n)$

```

#include<bits/stdc++.h>
using namespace std;
const int N=5e5+10;

int n,m;
vector<int>ve[N];
int sheep[N],fa[N],dep[N];

//计算每个点的深度和父节点
inline void dfs_tree(int u,int f){
    fa[u]=f,dep[u]=dep[f]+1;
    for(int i=0;i<ve[u].size();i++){
        int j=ve[u][i];
        if(j==f) continue;
        dfs_tree(j,u);
    }
}

//按照结点深度排序
bool cmp(int x,int y){
    return dep[x]>dep[y];
}

//计算每个点距离羊点的最近距离
queue<int>q;
int dist[N];

```

```

void bfs(){
    while(q.size()){
        int u=q.front(); q.pop();
        for(int i=0;i<ve[u].size();i++){
            int j=ve[u][i];
            if(dist[j]!=-1) continue;
            dist[j]=dist[u]+1;
            q.push(j);
        }
    }
}

//标记u的子树内的羊点
bool vis[N];
inline void dfs(int u,int d){
    vis[u]=1;
    for(int i=0;i<ve[u].size();i++){
        int j=ve[u][i];
        if(dist[j]!=d-1 || vis[j]) continue;
        dfs(j,d-1);
    }
}

int main(){

    cin>>n>>m;
    for(int i=0;i<n-1;i++){
        int a,b; cin>>a>>b;
        ve[a].push_back(b);
        ve[b].push_back(a);
    }
    dfs_tree(1,0); //计算每个点的父节点, 深度

    memset(dist,-1,sizeof dist);
    for(int i=0;i<m;i++){
        cin>>sheep[i];
        dist[sheep[i]]=0;
        q.push(sheep[i]);
    }
    bfs(); //计算每个点距离羊点的最近距离

    sort(sheep,sheep+m,cmp); //按照深度排序

    vector<int>ans;
    for(int i=0;i<m;i++){
        int now=sheep[i];
        if(vis[now]) continue;

```

```

        while(fa[now]&&dist[fa[now]]==dist[now]+1) now=fa[now]; //尽可能找最高的祖宗now
        dfs(now,dep[sheep[i]]-dep[now]); //标记now中的羊点，之后不再遍历
        ans.push_back(now);
    }

    cout<<ans.size()<<endl;
    for(int i=0;i<ans.size();i++) cout<<ans[i]<<' ';
    return 0;
}

```

J-线段值

线段树进行维护，记录线段树节点 $T[x][0/1][0/1]$ ，表示线段树节点为 x 所代表的区间能取到的区间最值。

后面的两个0/1 分别表示左端点的 d_i 取还是不取，以及右端点 d_i 取还是不取。修改时只需要修改所需要修改的区间的左端点以及右端点然后更新到线段树上即可。时间复杂度为 $O(q\log_2 n)$ 。

```

#include<bits/stdc++.h>

#define int long long
#define N 1000100
using namespace std;
int n,a[N],q,d[N];
int t[N*4][2][2]; //线段树
void update(int p,int num)
{
    for (int i=0;i<2;i++) for (int j=0;j<2;j++)
    {
        t[p][i][j]=-1e18;
        for (int k=0;k<2;k++) for (int l=0;l<2;l++) if
        (l!=1||k!=1||d[num]*d[num+1]>=0) t[p][i][j]=max(t[p][i][j],t[p*2][i][k]+t[p*2+1][l]
        [j]);
    }
} //更新线段树p节点上的值
void add(int p,int l,int r,int where)
{
    if (l==r)
    {
        t[p][1][1]=abs(d[l]);
        return;
    }
    int mid=l+r>>1;
    if (where<=mid) add(p*2,l,mid,where);
    if (where>mid) add(p*2+1,mid+1,r,where);
    update(p,mid);
} //更新线段树(单点修改)

```

```

int query(int x)
{
    return max(t[x][0][0],max(t[x][0][1],max(t[x][1][0],t[x][1][1])));
} // 查询线段树上x节点的最大值
signed main()
{
    cin>>n>>q;
    for (int i=1;i<=n;i++) cin>>a[i];
    for (int i=1;i<=n;i++) d[i]=i==1?0:a[i]-a[i-1],add(1,1,n,i); // 差分并插入线段树上
    while (q--)
    {
        int l,r,k;
        cin>>l>>r>>k;
        if (l!=1)
        {
            d[l]+=k;
            add(1,1,n,l);
        } // 添加左端点
        if (r!=n)
        {
            d[r+1]-=k;
            add(1,1,n,r+1);
        } // 添加右端点
        cout<<query(1)<<endl; // 查询[1,n]的区间最值
    }
    return 0;
}

```